

灵犀（LingXi）智能客服

- ☐ 1. 项目背景与实现功能介绍
- ☐ 2. 项目整体架构层级设计
- ☐ 3. 底层技术栈与核心逻辑说明
- ☐ 4. 本地私有化部署与启动流程
- ☐ 5. 结合电商场景的五大业务模块功能分类
- ☐ 6. Multi-Agent 架构下的图设计与数据流向
- ☐ 7. 用户意图识别模块实现思路
- ☐ 8. 安全护栏（Guardrails）模块实现思路
- ☐ 9. LangGraph 实现复杂/多跳任务分解模块
- ☐ 10. 多工具统一接口规范与自定义工具接入方法
- ☐ 11. 核心组件：混合知识库检索子图构建思路
- ☐ 12. 图片识别接入与 LangGraph 图状态集成

项目背景与实现功能介绍

智能客服**灵犀**案例是我们课程完全自主研发的一个基于大模型的落地应用项目，整体构建的是一个面向企业级用户的高效对话、咨询服务的解决方案。从项目整体架构上，采用的是前后端分离架构设计，前端基于 `Vue3` 框架实现交互式界面，后端通过 `Python FastAPI` 提供标准化 `RestFul API` 接口。实现形式类似主流电商平台（如京东、淘宝）智能客服功能，具备多场景意图识别、混合检索问答、动态上下文理解等核心能力，旨在实现用户问题的高效响应与精准解答。

从**灵犀**实现的功能来看，该项目主要围绕电商垂直领域进行设计和研发，可以根据用户输入的实时售前、售后问题，自动匹配到对应的业务流程，并根据业务流程，自动调用对应的业务组件，完成对用户问题的精准解答，其中核心的业务组件包括了：



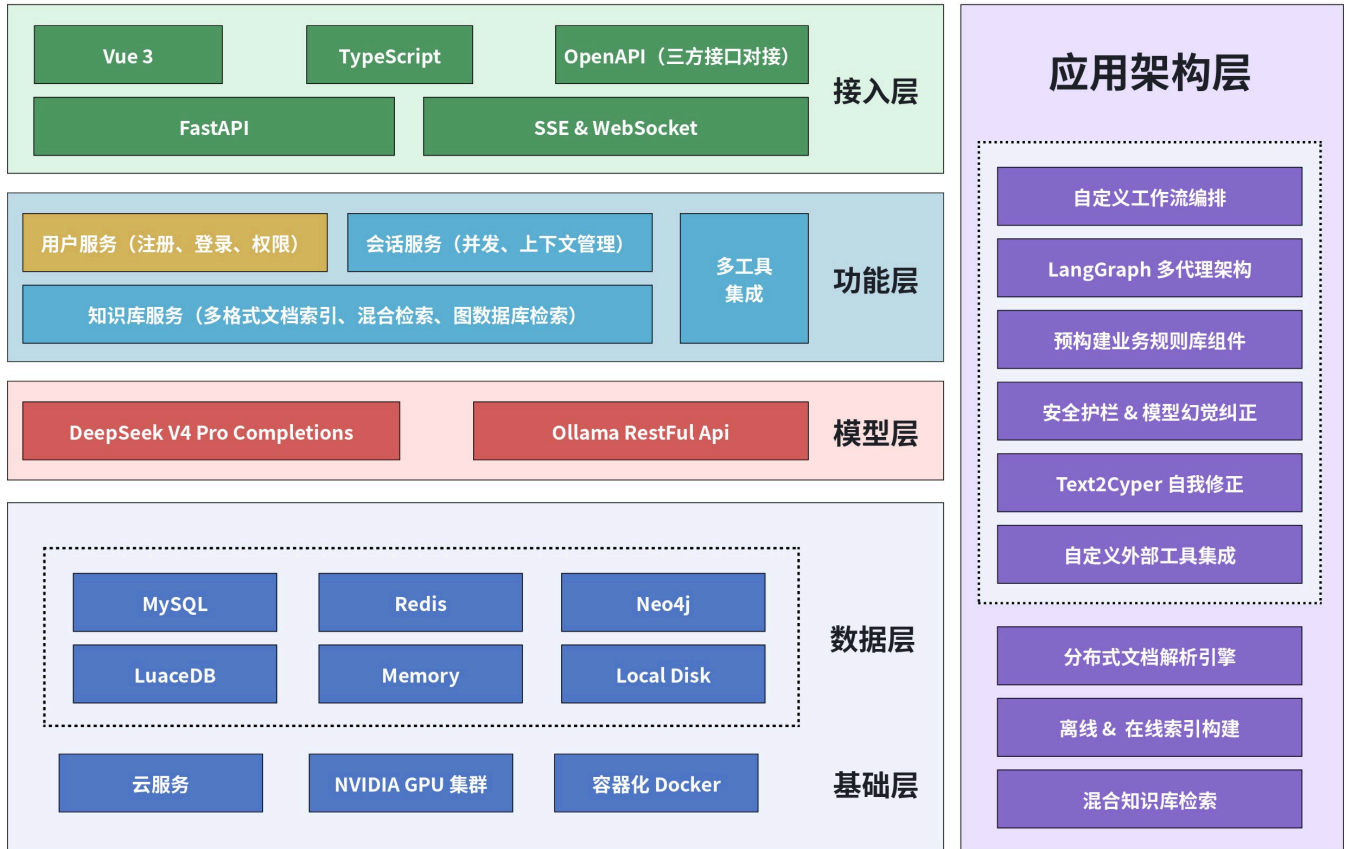
（注意：以上业务组件在适配自己的业务和需求时，均可以快速进行替换和调整）

项目实现了大模型在传统业务流程中的有效接入，同时依托于目前最主流的 `Multi-Agent` 架构设计，使复杂且不确定性的业务需求可以通过统一的 `API` 接口完成用户无感知的自动处理，包括了用户的问题是否与自有业务相关、何时调用本地知识库数据进行查询、如何避免大模型因幻觉问题产生胡言论语等常见AI服务痛点。

项目整体架构层级设计

灵犀在底层的架构实现上做了大量优化与创新，同时为了方便今后更简单、快速迁移到自己业务中，对核心组件进行了高度抽象和封装，并提供了丰富的 `API` 接口，方便进行二次开发和适配。因此，完全可以将**灵犀**项目作为自己业务系统中的一个独立模块，进行快速集成和使用。**灵犀**项目整体架构层级图如下图所示：

灵犀（LingXi）系统架构层级



从架构上看，**灵犀**底层通过深度的工程化集成，将异构技术栈整合为一套标准化、高复用、易扩展的综合性应用系统，致力于打造企业级快速开发的基础底座。各个层级对应的技术栈如下：

- 模型集成：**集成 DeepSeek V4 Pro （可提示）的在线 API 接口以及通过 Ollama 启动的开源模型（只要是 Ollama 支持的模型都可以）接入；
- AI Agent 开发框架：**采用了 LangGraph 构建的 Multi-Agent 架构，并结合 LangChain 的组件化设计接入模型的集成功能；
- 意图识别：**通过 LangChain 的 PromptTemplate + Chain 组件，接入 LangGraph 的图结构中，实现用户的意图识别功能；
- 本地知识库构建：**通过 Microsoft GraphRAG 组件，实现本地知识库的索引和查询，并作为子图接入到 LangGraph 的图结构中；
- Neo4j 图数据库检索：**通过预构建规则库 + 大模型根据用户意图自动生成 Cypher 语句，实现结构化知识库的检索功能，作为 LangGraph 的子图接入到图结构中；
- 多工具集成：**通过 LangChain 的 Tool 组件，采用 LangGraph 图结构的 Map-Reduce 分支，实现多工具的并行调用，并根据任务分解后的子任务进行结果的汇总和处理；
- FastAPI 接口服务：**通过 FastAPI 框架，对编译后的 LangGraph 图结构进行接口封装，提供与外部系统对接的标准化 RestFul API 接口服务；

从上述技术栈中可以看出，**灵犀**核心底层采用的是 AI Agent 开发框架 `LangGraph` 与 RAG 开发框架 `Microsoft GraphRAG`

本地私有化部署

本地进行私有化部署时，只需要进行项目代码的下载，安装项目依赖，最后完成项目配置文件的修改，即可快速启动项目应用。但因为项目中涉及到非常多的功能和组件，因此配置项非常多，下文是在部署过程中涉及到的配置项逐一展开详细的说明。

1. 下载源码

代码块

```
1 git@github.com:Shichuan-Hao/lingxi_backend.git
```

2. 使用 IDE 打开项目

3. 重建虚拟环境

方式一：Python venv（轻量）*

```
1 # Windows
2 python -m venv .venv
3 Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
4 .venv\Scripts\activate
5
6 # Linux / Mac
7 python -m venv .venv
8 source .venv/bin/activate
```

方式二：Conda（推荐）

```
1 conda create -n lingxi python=3.11 -y
2 conda activate lingxi
```

4. 安装依赖

安装依赖

```
1 pip install -r requirements.txt
2 # 使用国内镜像加速（可选）
3 pip install -r requirements.txt -i https://pypi.tuna.tsinghua.edu.cn/simple
```

5. 修改项目配置文件。将项目跟路径下的 `.env_example` 复制为 `.env`，相关配置如下：

代码块

```
1  # =====
2  # Deepseek 云端 API 配置（用于调用官方模型服务）
3  # =====
4
5  # 作用：Deepseek 官方 API 的身份验证密钥（相当于账号密码）
6  # 使用场景：每次调用 API 时，程序会在请求头中携带此密钥进行鉴权
7  # 获取方式：去 https://platform.deepseek.com/ 注册账号后申请
8  # 注意：示例中的 sk-... 是假的，必须替换成你自己的真实密钥
9  DEEPSEEK_API_KEY=sk-107e3e19f4b047e691ef47451661b6d6fdasfa
10
11 # 作用：Deepseek API 服务的入口地址
12 # 使用场景：程序拼接完整请求 URL 时使用（BASE_URL + /chat/completions 等）
13 # 注意：如果你使用官方服务，保持默认即可；如果是代理/网关，改成对应地址
14 DEEPSEEK_BASE_URL=https://api.deepseek.com
15
16 # 作用：指定调用 Deepseek 的哪个模型
17 # 可选值：deepseek-v4-pro（旗舰版）、deepseek-v4-flash（轻量版）、deepseek-r1（推理版）等
18 # 注意：当 CHAT_SERVICE=deepseek 或 REASON_SERVICE=deepseek 时，此配置生效
19 DEEPSEEK_MODEL=deepseek-v4-pro
20
21
22 # =====
23 # Ollama 本地模型配置（用于本地离线推理）
24 # =====
25
26 # 作用：Ollama 本地服务的访问地址和端口
27 # 使用场景：程序需要调用本地模型时，会向这个地址发送请求
28 # 默认值：http://localhost:11434（Ollama 安装后的默认监听地址）
29 # 注意：运行项目前，务必确保 Ollama 服务已启动（ollama serve 或桌面版开启）
30 OLLAMA_BASE_URL=http://localhost:11434
31
32 # 作用：指定用 Ollama 处理聊天任务时，调用哪个模型
33 # 可选值：任意已下载的模型，如 qwen2.5:7b、llama3.2:3b、deepseek-r1:7b 等
34 # 当前值：qwen2.5:1.5b（小模型，响应快但能力有限）
35 # 注意：模型必须先用 ollama pull 模型名 下载到本地，否则会报错
36 OLLAMA_CHAT_MODEL=qwen2.5:1.5b
37
38 # 作用：指定用 Ollama 处理推理/思考任务时，调用哪个模型
39 # 当前值：deepseek-r1:7b（专门的推理模型，逻辑分析能力强）
40 # 注意：当 REASON_SERVICE=ollama 时，此配置生效
41 OLLAMA_REASON_MODEL=deepseek-r1:7b
42
43
44 # =====
```

```
45 # 服务路由开关 (决定用云端还是本地)
46 # =====
47
48 # 作用: 决定聊天任务用哪个服务提供方
49 # 可选值: deepseek (云端 API) 或 ollama (本地模型)
50 # 当前值: deepseek — 日常聊天走云端, 效果好
51 CHAT_SERVICE=ollama
52 #CHAT_SERVICE=deepseek
53
54 # 作用: 决定推理/思考任务用哪个服务提供方
55 # 可选值: deepseek (云端 API) 或 ollama (本地模型)
56 # 当前值: ollama — 复杂推理走本地 deepseek-r1:7b, 省钱且保护隐私
57 REASON_SERVICE=ollama
58
59
60 # =====
61 # SerpAPI 搜索增强配置 (预留, 本期未使用)
62 # =====
63
64 # 作用: SerpAPI 密钥, 用于调用搜索引擎获取实时信息
65 # 使用场景: 现联网检索功能, 届时需要替换为真实密钥
66 # 注意: 本期代码不会用到这个配置, 可以暂时保留占位值
67 SERPAPI_KEY=your-serpapi-key
68
69 # =====
70 # 数据库配置 (用于存储用户、对话、消息记录)
71 # =====
72
73 # 作用: MySQL 服务器地址
74 # 当前值: localhost (本机安装的 MySQL)
75 DB_HOST=192.168.1.9
76
77 # 作用: MySQL 服务端口, 默认 3306
78 DB_PORT=3306
79
80 # 作用: 数据库登录用户名
81 # 注意: 替换为你实际的 MySQL 用户名
82 DB_USER=root
83
84 # 作用: 数据库登录密码
85 # 注意: 替换为你实际的 MySQL 密码
86 DB_PASSWORD=123456
87
88 # 作用: 使用的数据库名称
89 # 注意: 启动前需要先在 MySQL 中创建这个库 (见启动文档)
90 DB_NAME=ling_xi
91
```

```

92
93 # =====
94 # JWT 认证配置 (用于用户登录 Token)
95 # =====
96
97 # 作用: 签发和校验 JWT Token 的密钥 (相当于签名私钥)
98 # 生成方式:
99 # 方式1 (推荐) - Python 一行生成:
100 #     python -c "import secrets; print(secrets.token_hex(32))"
101 # 方式2 - 在命令行使用 openssl:
102 #     openssl rand -hex 32
103 # 注意:
104 # - 生产环境务必换成随机生成的密钥, 不要用默认值
105 # - 密钥泄露后, 任何人都可以伪造登录 Token, 需要立即更换
106 # - 密钥长度建议 ≥ 32 字节 (64 位十六进制字符)
107 SECRET_KEY=51ae1a4ab84b4a1dd733f7f0130bb03656d5034c393f0abae446d4f29a2b2635
108
109 # 作用: JWT 签名算法
110 # 默认值: HS256 (HMAC-SHA256, 对称加密, 性能好适合单体应用)
111 # 可选值: HS256、HS384、HS512、RS256 (非对称) 等
112 ALGORITHM=HS256
113
114 # 作用: 登录 Token 的有效期 (分钟)
115 # 当前值: 1440 (即 24 小时), 过期后用户需要重新登录
116 # 调整建议: 安全敏感场景设 30 分钟, 普通应用设 1440 分钟
117 ACCESS_TOKEN_EXPIRE_MINUTES=1440

```

其相关配置说明规则如上所示, 其中对智能客服场景的统一接口服务, 选择的配置是

AGENT_SERVICE

6. 启动项目

开发模式

```
1 python run.py
```

生产模式 (关闭热加载)

```

1 # 单 Worker
2 python run.py --prod
3
4 # 多 Worker
5 python run.py --prod -w 4

```



注意：在 `run.py` 中可以指定访问服务的 IP 地址和端口号，默认是 `0.0.0.0:8000`，大家可以根据实际情况进行修改。启动后，即可在对应的浏览器中访问服务。如果是本地启动，则可以访问 `http://localhost:8000`，如果是服务器启动，则可以访问 `http://<服务器IP>:8000`。

启动页面如下：

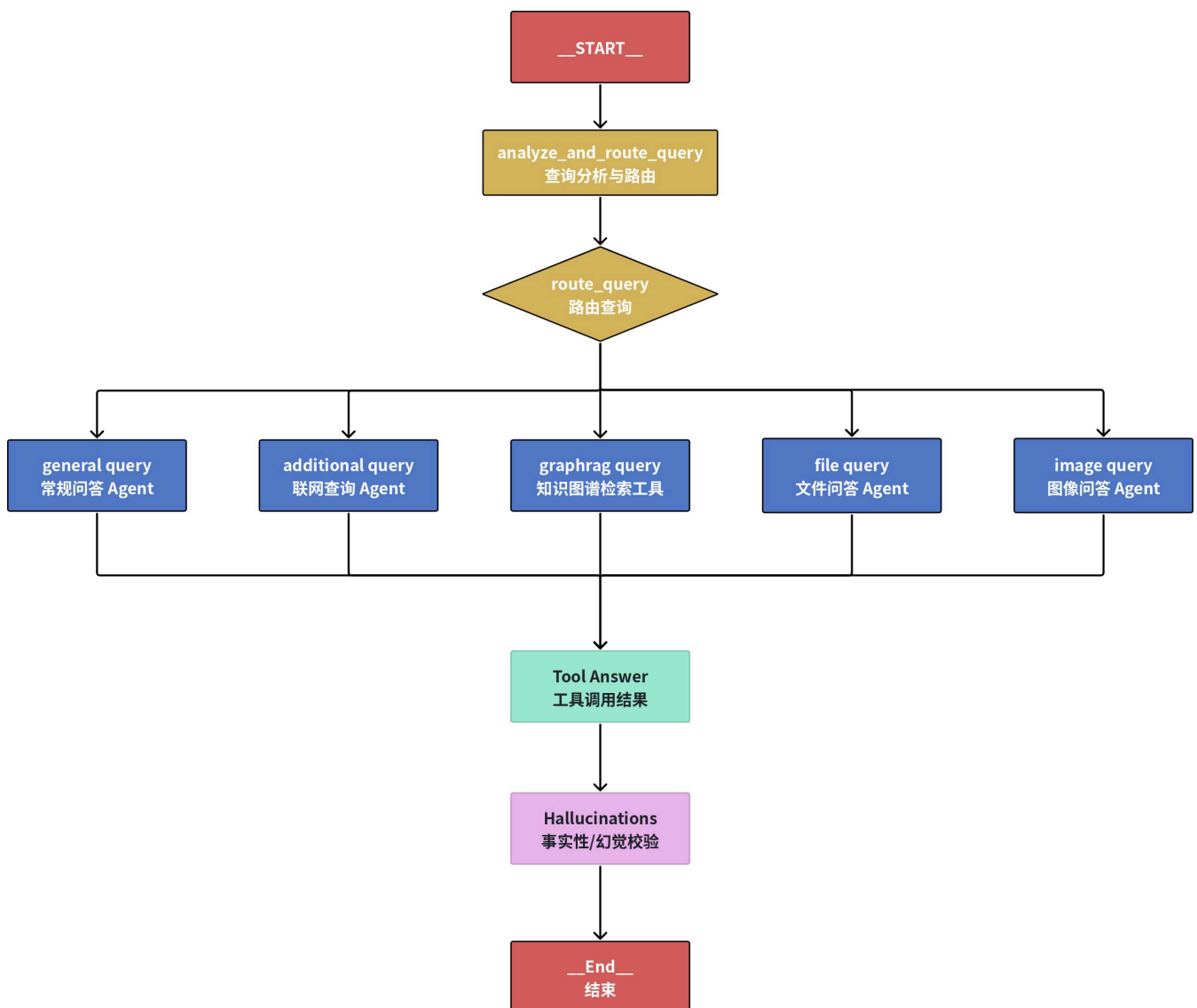
点击 `电商客服` 按钮，即可进入电商智能客服的测试页面，如下图所示：

在测试页面中，可以点击 `联系客服` 按钮开始进行智能客服的问答。大家可以自行输入问题进行尝试。

核心模块详解

本项目整体采用的是一个 `Multi-Agent` 架构，并基于 `LangGraph` 框架构建，是对 `LangGraph` 完整技术体系做的工程化集成开发，包含但不限于：**节点和边的定义、图状态的定义、事件流、工具集成、上下文记忆、路由、父子图**等核心组件的使用。除此以外，项目中还多处涉及到**提示工程**以及与 `LangChain` 组件的结合使用。综上，结合智能客服场景，`AssistGen` 项目底层的 `Multi-Agent` 架构如下图所示：

Multi Agent Architecture



如上图所示，在 `langgraph` 框架的架构下，多代理的实现主要是 `Router` 组件来实现的，完整的功能链路则是通过节点和边来进行串联。其中：

- 由 `analyze_and_route_query` 和 `route_query` 两个节点充当意图识别模块，根据用户输入的问题，进行意图识别，并根据识别结果，将问题路由到对应的子图进行处理；
- 由 `general-query`、`additional-query`、`graphrag-query`、`image-query`、`file-query` 五个节点充当业务处理模块，根据意图识别的结果进行对应的业务处理，并将处理结果返回给用户，其中
 - `general-query` 节点负责处理一般性问题，即与商品、订单、售后、技术支持无关的闲聊问题；
 - `additional-query` 节点负责处理需要更多信息才能回答的问题，比如用户询问商品但没有提供具体型号或规格、用户询问订单状态但没有提供订单号等；

- `graphrag-query` 节点负责处理需要查询知识库的问题，比如用户询问商品价格、库存、规格等信息查询、订单状态、物流信息查询、会员积分、优惠活动查询、退换货政策咨询、商品使用指导及故障解决方法等；
- `image-query` 节点负责处理需要解析用户上传图片的问题；
- `file-query` 节点负责处理需要解析用户上传文件的问题；
- `Hallucinations` 节点则作为最后一步，负责对最终要返回给用户的问题进行幻觉检测，如果检测到问题存在幻觉，则进行修正，并返回给用户。

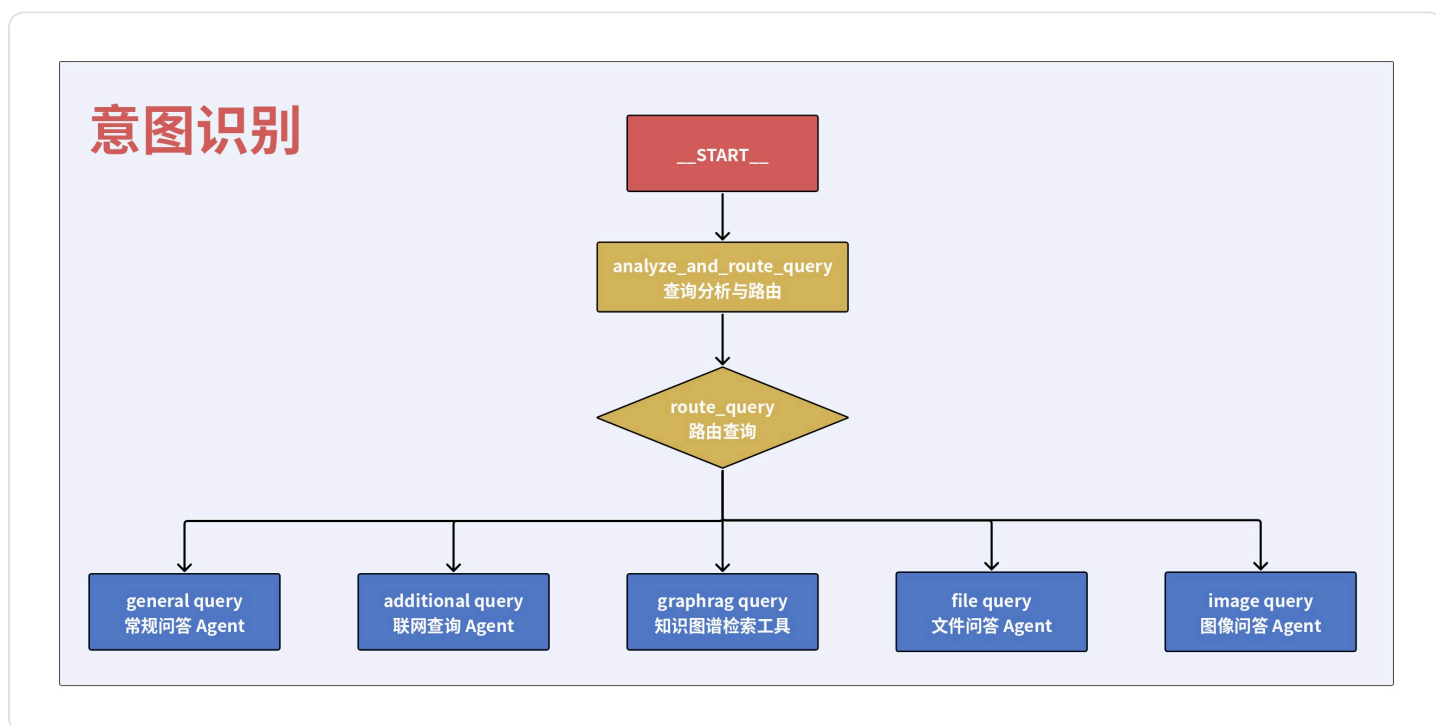
以上是整体 `Multi-Agent` 架构的路由，每个子图的实现细节见下文。

子路由的核心模块是 `graphrag-query`，负责处理需要查询知识库的问题，包括了对 `Neo4j` 图数据库数据的查询和对 `Microsoft GraphRAG` 的查询，每种查询方式的实现思路完全不同，但借助 `LangGraph` 的路由组件，可以自动的将问题路由到对应的子图进行处理，并根据处理结果返回给用户。上述流程也是目前基于 `LangGraph` 构建底层 `Agent` 服务的主流实现方式。

意图识别模块

意图识别模块是任何一个基于大模型构建的应用系统都需要重点关注的，所谓的**用户意图识别**，简单理解就是**明确用户输入的问题到底要解决什么问题**。用户输入的问题一定会是千奇百怪，如果不能进行有效的意图识别，则无法将问题路由到对应的子图进行处理，也就无法给出准确的回答。比如用户询问商品价格，那么意图识别模块就需要将这个问题识别为商品价格查询，然后路由到对应的子图进行处理。

因此，意图识别模块是 `Multi-Agent` 架构中非常重要的一环。但实际在实现过程中，借助 `LangGraph` 的路由组件，可以非常方便的实现意图识别功能，该模块在完整的 `Multi-Agent` 架构中处于第一层级，直接接收用户输入的原始问题，并分派给对应的子图进行处理，如下图所示：



该模块在 `LangGraph` 的 `Router` 组件中实现，具体的源码实现为 `analyze_and_route_query` 节点和 `route_query` 节点搭配实现。

意图识别模块在 `langGraph` 中的实现关键是根据**提示工程 + 结构化输出**。

提示工程的有效方法是**要对每一个子路由能处理的问题进行详细的描述，从而引导大模型给出正确的分类**。比如对于电商智能客服场景，我设计的提示工程如下：

代码块

```
1  ROUTER_SYSTEM_PROMPT = """你是一个电商领域的智能客服。你的工作是帮助用户解决与产品、订
   单、售后和技术支持相关的问题。
2
3  用户会向你提出询问。你的首要任务是对询问类型进行分类。你必须将用户询问的问题分类为以下类型
   之一：
4
5  ## `general-query`
6  如果是一般性问题，不需要查询知识库就能回答，请将其分类为此类。
7  包括但不限于：
8  - 与商品、订单、售后、技术支持无关的闲聊问题
9
10 ## `additional-query`
11 如果你需要更多信息才能帮助用户，请将用户询问分类为此类。例如：
12 - 用户询问商品但没有提供具体型号或规格
13 - 用户询问订单状态但没有提供订单号
14 - 用户描述问题不够具体，无法提供准确帮助
15
16 ## `graphrag-query`
17 如果通过查询本地知识库可以回答用户询问，请将其分类为此类。
18 包括但不限于：
19 - 商品价格、库存、规格等信息查询
20 - 订单状态、物流信息查询
21 - 会员积分、优惠活动查询
22 - 退换货政策咨询
23 - 商品使用指导及故障解决方法
24
25 ## `image-query`
26 如果用户提供了图片请求提供图片，请将其分类为此类。
27
28
29 ## `file-query`
30 如果用户上传了文件，请将其分类为此类。
31 """
```

结构化输出，则是借助 `LangChain` 的 `with_structured_output` 方法，将大模型的输出结果转换为具体的结构化数据，并通过 `langGraph` 定义图的输出类型，生成结构化的输出结果。伪代码

如下图所示：

代码块

```
1  # 定义路由的输出类型
2  class Router(TypedDict):
3  """Classify user query."""
4  logic: str
5  type: Literal["general-query", "additional-query", "graphrag-query", "image-
    query", "file-query"]
6  question: str = field(default_factory=str)
7
8  # 使用LangChain的with_structured_output方法将大模型的输出结果转换为具体的结构化数据
9  response = cast(
10     Router, await model.with_structured_output(Router).ainvoke(messages)
11 )
```

通过上述代码，经过意图识别模块处理后，得到的结果只会是 `type: general-query`、`additional-query`、`graphrag-query`、`image-query`、`file-query` 中的一种，从而可以非常方便的将问题路由到对应的子图进行处理。然后，再交由 `route_query` 节点，找到对应的子图，准备进行下一步处理。

代码块

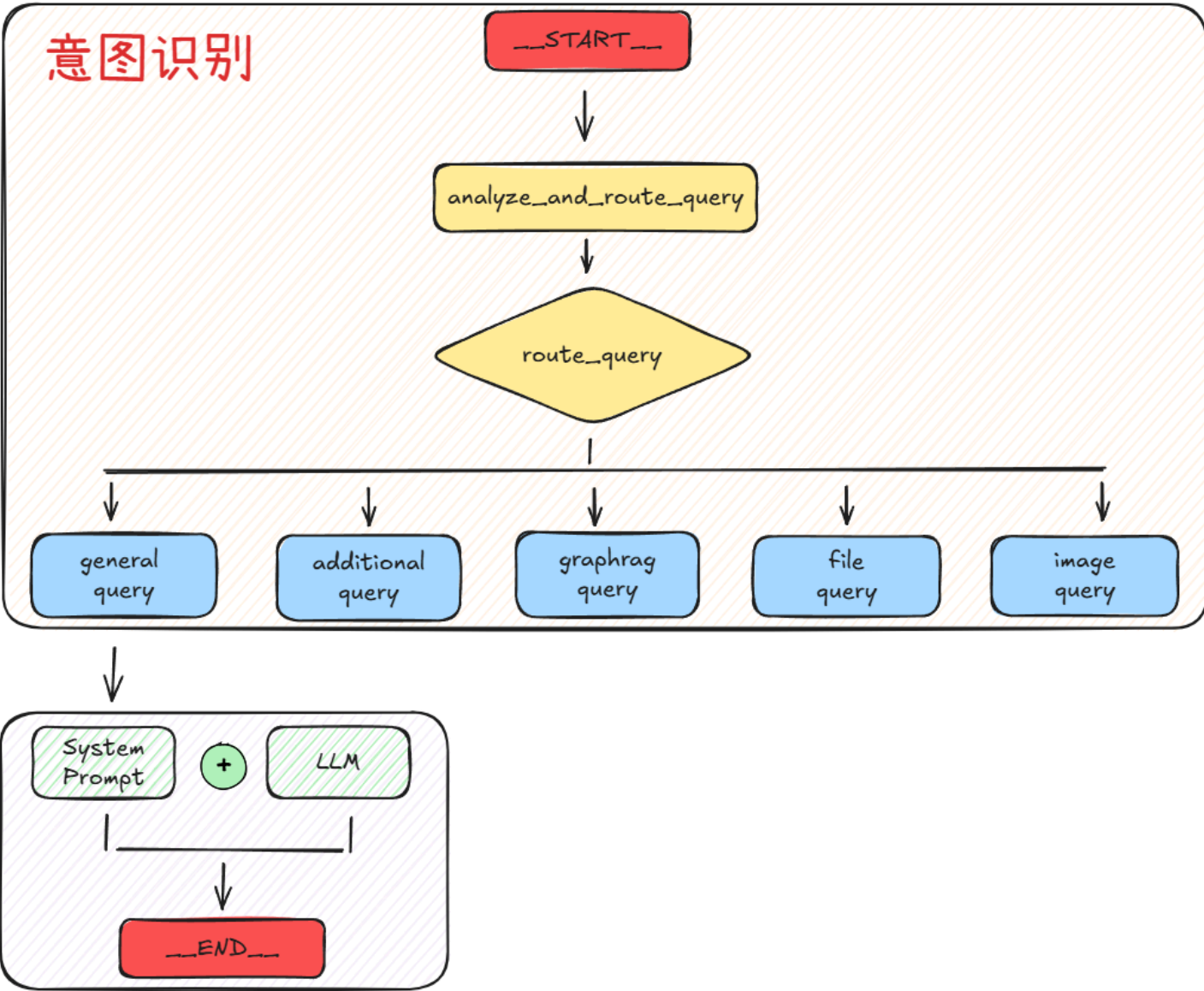
```
1  def route_query():
2      if _type == "general-query":
3          return "respond_to_general_query"
4      elif _type == "additional-query":
5          return "get_additional_info"
6      elif _type == "graphrag-query":
7          return "create_research_plan"
8      elif _type == "image-query":
9          return "create_image_query"
10     elif _type == "file-query":
11         return "create_file_query"
```

注意：这里的 `respond_to_general_query`、`get_additional_info`、`create_research_plan`、`create_image_query`、`create_file_query` 需要自定义的节点和子图，每个节点或子图，需要根据具体的业务需求进行实现。

接下来下文依次是每个节点/子图的实现细节介绍。

图节点 `general-query` 实现逻辑

`general-query` 节点负责处理一般性问题，即与商品、订单、售后、技术支持无关的闲聊问题。如下图所示：



该节点的实现思路并不复杂，可以直接根据提示工程即可实现，所以这个节点的关键就是需要在提示工程中，对“闲聊、一般性”问题应该如何回复进行详细的描述，这就要根据具体的业务需求进行设计。比如对风格、语气、回复策略等进行详细的描述。如下所示：

代码块

```
1 GENERAL_QUERY_SYSTEM_PROMPT = """你是一个电商领域的智能客服。你的工作是帮助用户解决与
  产品、订单、售后和技术支持相关的问题。
2
3 请以类似淘宝/京东等知名电商客服的风格回复用户，遵循以下规则：
4
5 ## 基本礼仪
6 1. 开场必须用"亲~"或"顾客您好~"问候
7 2. 使用积极、温暖的语气
8 3. 适当使用emoji表情（如 🙌 😊 ❤️）增加亲和力
9 4. 结尾必须表达感谢和继续服务的意愿
```

```

10
11  ## 回复策略
12  如果用户问题模糊不清：
13  1. 先表示理解和重视："感谢您的咨询～"
14  2. 友好地指出不明确之处："不过亲，为了更好地帮助您..."
15  3. 引导用户提供更具具体信息："您能告诉我具体是..."
16  4. 给出一些示例："比如说，您是想问..."
17
18  ## 如果问题与电商业务无关：
19  1. 先表示感谢："感谢您的咨询～"
20  2. 委婉说明情况："不好意思呢亲，这个问题可能不太属于我们的业务范围..."
21  3. 建议其他帮助渠道："您可以尝试咨询相关专业的平台/机构..."
22  4. 表达歉意和继续服务意愿："如果您有任何关于我们商品或服务的问题，随时欢迎询问哦～"
23
24  ## 回复要点
25  - 保持专业性：提供清晰、准确的信息
26  - 给出建议：在可能的情况下提供实用的解决方案
27  - 态度温和：即使是拒绝也要保持友好
28  - 预设期待：为后续可能的咨询留下互动空间
29
30  ## 示例回复格式
31
32  ### 模糊问题示例：
33  "亲～感谢您的咨询～😊 为了能更准确地帮助您，麻烦您能告诉我具体是哪款商品呢？这样我才能...
    （根据具体情况询问）"
34
35  ### 无关问题示例：
36  "亲～感谢您的咨询～😊 非常抱歉呢，这个问题可能不太属于我们的业务范围，建议您可以...（给出
    建议）。如果您有任何关于我们商品或服务的问题，随时欢迎询问哦～❤️"
37
38  系统已确定用户正在提出一般性问题，不需要查询特定数据库即可回答。以下是分类理由：
39
40  <logic>
41  {logic}
42  </logic>
43
44  记住：无论是哪种情况，都要保持亲切、专业的语气，让用户感受到被重视和理解。
45  同时，尽量保持回复的简洁性，不要过于冗长，尽量限制在20个字以内。
46  ""

```

直接将上述提示工程作为 `System Message` 传递给大模型，并将其结果存储到 `langGraph` 的全局状态中即可。伪代码如下所示：

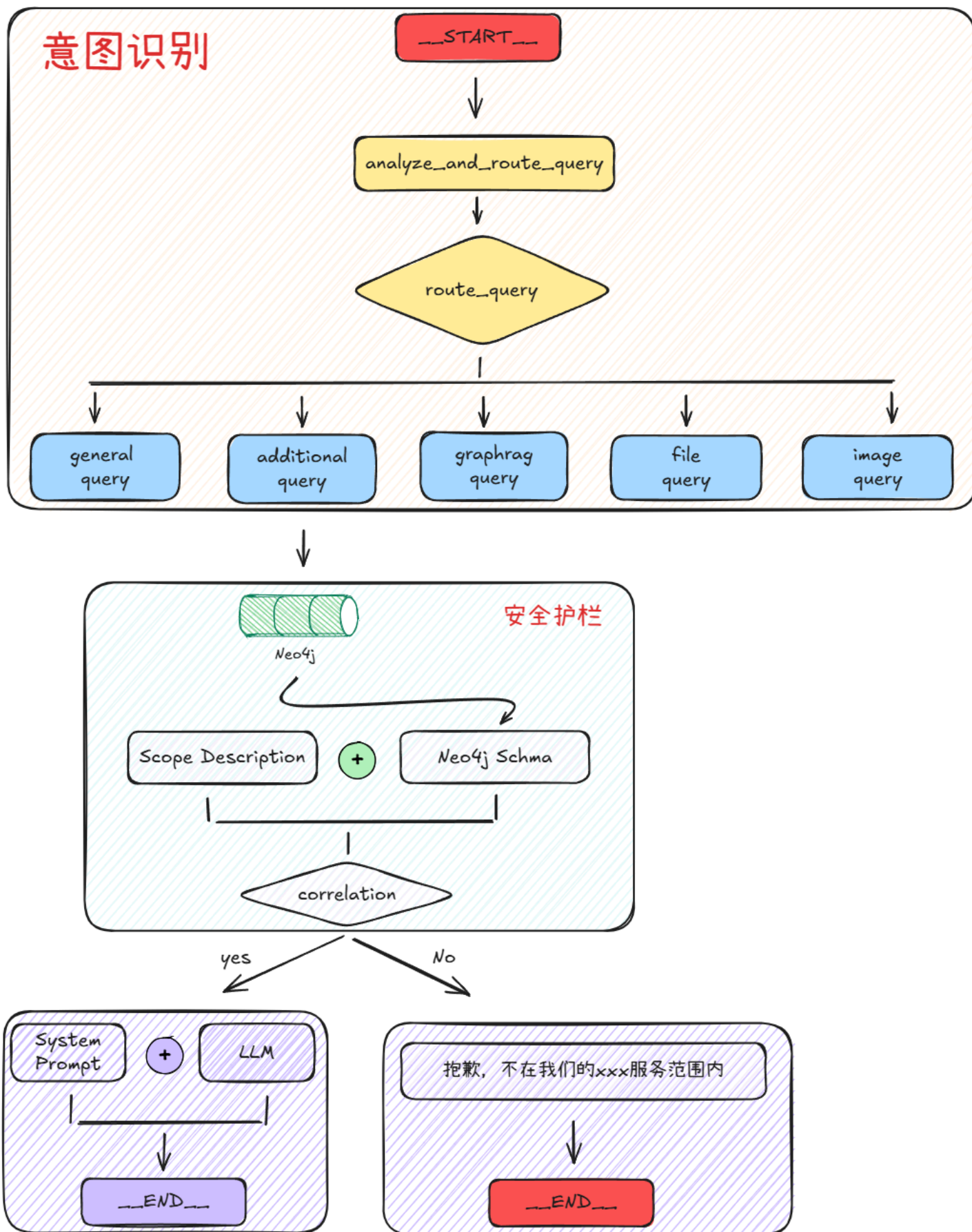
代码块

```
1  if settings.AGENT_SERVICE == ServiceType.DEEPSEEK:
```

```
2     model = ChatDeepSeek(api_key=settings.DEEPSEEK_API_KEY,  
    model_name=settings.DEEPSEEK_MODEL, temperature=0.7, tags=["general_query"])  
3 else:  
4     model = ChatOllama(model=settings.OLLAMA_AGENT_MODEL,  
    base_url=settings.OLLAMA_BASE_URL, temperature=0.7, tags=["general_query"])  
5  
6     system_prompt = GENERAL_QUERY_SYSTEM_PROMPT.format(  
7         logic=state.router["logic"]  
8     )  
9  
10    messages = [{"role": "system", "content": system_prompt}] + state.messages  
11    response = await model.ainvoke(messages)  
12    return {"messages": [response]}
```

图节点 `additional-query` 实现逻辑

`additional-query` 节点负责处理需要更多信息才能回答的问题，比如用户询问商品但没有提供具体型号或规格、用户询问订单状态但没有提供订单号等，这种情况下是没有办法去本地的数据库或者知识库中查询出有效结果的，所以需要引导用户提供更多支撑知识库检索的必要信息。其实现逻辑如下图所示：



首先第一个处理组件是安全护栏，其核心作用是用来判断用户的问题是否属于服务的范围。比如我们的商品只有智能家居类，但是用户要买服装类，那么这个请求就属于超出服务范围的。因为 `general-query` 处理的是闲聊问题，所以只要用户提问的问题与电商相关，那么就一定不会进入 `general-query` 节点。但是可能存在的问题是：用户即使咨询的是电商问题，但其实和我们的业

务是没有关系的。针对这种情况，安全护栏起到的作用就是：先去判断用户的问题是否属于个人服务的范围，如果属于，再继续追问，否则便可以直接回复特定的信息，比如“抱歉，您咨询的问题和我们的业务范围不符，请您咨询其他平台”等，直接结束掉当前会话。

如何判断用户的问题是不是属于经营范围呢？这就需要大家根据具体的业务需求进行设计了。但能够确定的是：提示工程是通用的，通过提示词明确的告诉大模型，哪些问题属于经营范围，哪些问题属于超出经营范围的。比如我设计的提示词如下所示：

代码块

```
1  scope_description = """
2  个人电商经营范围：智能家居产品，包括但不限于：
3  - 智能照明（灯泡、灯带、开关）
4  - 智能安防（摄像头、门锁、传感器）
5  - 智能控制（温控器、遥控器、集线器）
6  - 智能音箱（语音助手、音响）
7  - 智能厨电（电饭煲、冰箱、洗碗机）
8  - 智能清洁（扫地机器人、洗衣机）
9
10 不包含：服装、鞋类、体育用品、化妆品、食品等非智能家居产品。
11  """
```

其二，除了人工编写提示词，还可以灵活的根据业务情况、实现的架构，去找到一些动态的补充信息。比如我们这个项目会使用 `Neo4j` 图数据库去存储商品信息，那 `Neo4j Schema` 就是一个非常好的动态提示词。因为图数据的 `Schema` 中的 `Node` 和 `Relationship`、`Property` 其实反映的就是存储的私有数据的结构，它可以一定程度上体现出商品的分类、商品的属性、商品的价格、商品的库存、商品的供应商、商品的客户、商品的订单、商品的员工、商品的物流等信息，以帮助大模型整体去判断用户的问题是否属于个人经营范围。伪代码如下所示：

代码块

```
1  def retrieve_and_parse_schema_from_graph_for_prompts(graph: Neo4jGraph) -> str:
2
3      """
4      关键点：
5      schema 指的是 Neo4j 数据库的结构描述，包括：
6      - 节点类型：如 Product, Category, Supplier 等
7      - 节点属性：如 ProductName, UnitPrice, CategoryName 等
8      - 关系类型：如 BELONGS_TO, SUPPLIED_BY, CONTAINS 等
9      - 关系属性：关系上可能的属性（如有）
10
11     提取出来的Schema 大致如下：
12     Node properties:
13     - **Product**: ProductID, ProductName, UnitPrice, UnitsInStock...
```



```

14         - **Category**: CategoryID, CategoryName, Description...
15
16     Relationship properties:
17         - **BELONGS_TO**:
18         - **SUPPLIED_BY**:
19
20     必要性：
21     1. 动态适应数据库变化：如果数据库结构变化（新增节点类型、关系或属性），系统无需修改代码即可适应
22     2. 提高查询准确性：通过向大语言模型提供准确的数据库结构，大大降低生成错误查询的可能性
23     3. 促进零样本学习：即使没有特定领域的示例，模型也能根据提供的结构信息生成符合语法的查询
24     """
25
26     schema: str = graph.get_schema
27
28     # 过滤掉对用户查询不相关的内部结构信息
29     if "CypherQuery" in schema:
30         schema = re.sub(
31             get_cypher_query_node_graph_schema(), r"\2", schema,
32             flags=re.MULTILINE
33         )
34
35     # 在这里添加一行：将所有花括号替换为方括号，避免模板变量冲突
36     # 因为 Schema 中包含 { } ，会与 ChatPromptTemplate 模版中的 input_variables
37     schema = schema.replace("{", "[").replace("}", "]")
38
39     return schema

```

Neo4j Schema 能够帮助我们匹配用户问题与知识库结构（如商品价格、库存等信息的查询模式），但它确实无法直接告诉你商品的领域归属。Schema 主要提供的是数据结构信息，比如实体之间的关系、属性类型等，而不是具体的商品类别内容。例如，Neo4j Schema 一般会呈现这样的结构：

代码块

```

1 (Product)-[:HAS_PRICE]->(Price)
2 (Product)-[:HAS_INVENTORY]->(Inventory)
3 (Product)-[:BELONGS_TO]->(Category)

```

但这个结构本身并不能告诉你哪些具体的商品类别（如篮球鞋或智能灯泡）属于你的经营范围。比如当用户询问篮球鞋时，系统可能会根据 Schema 理解用户是在查询商品信息，但无法仅凭 Schema 判断篮球鞋是否在你的经营范围。所以，需要的就是人工编写的提示工程。两者结合便可以构建出一个非常完善的判断逻辑。当然，有了判断逻辑还不够，安全护栏作为一个 LangGraph 的

节点，还需要一个最终提示，引导大模型在结束安全护栏组建后输出结构化的结果，从而让后续的节点，能够根据安全护栏的输出结果，执行不同分支的代码。伪代码如下所示：

代码块

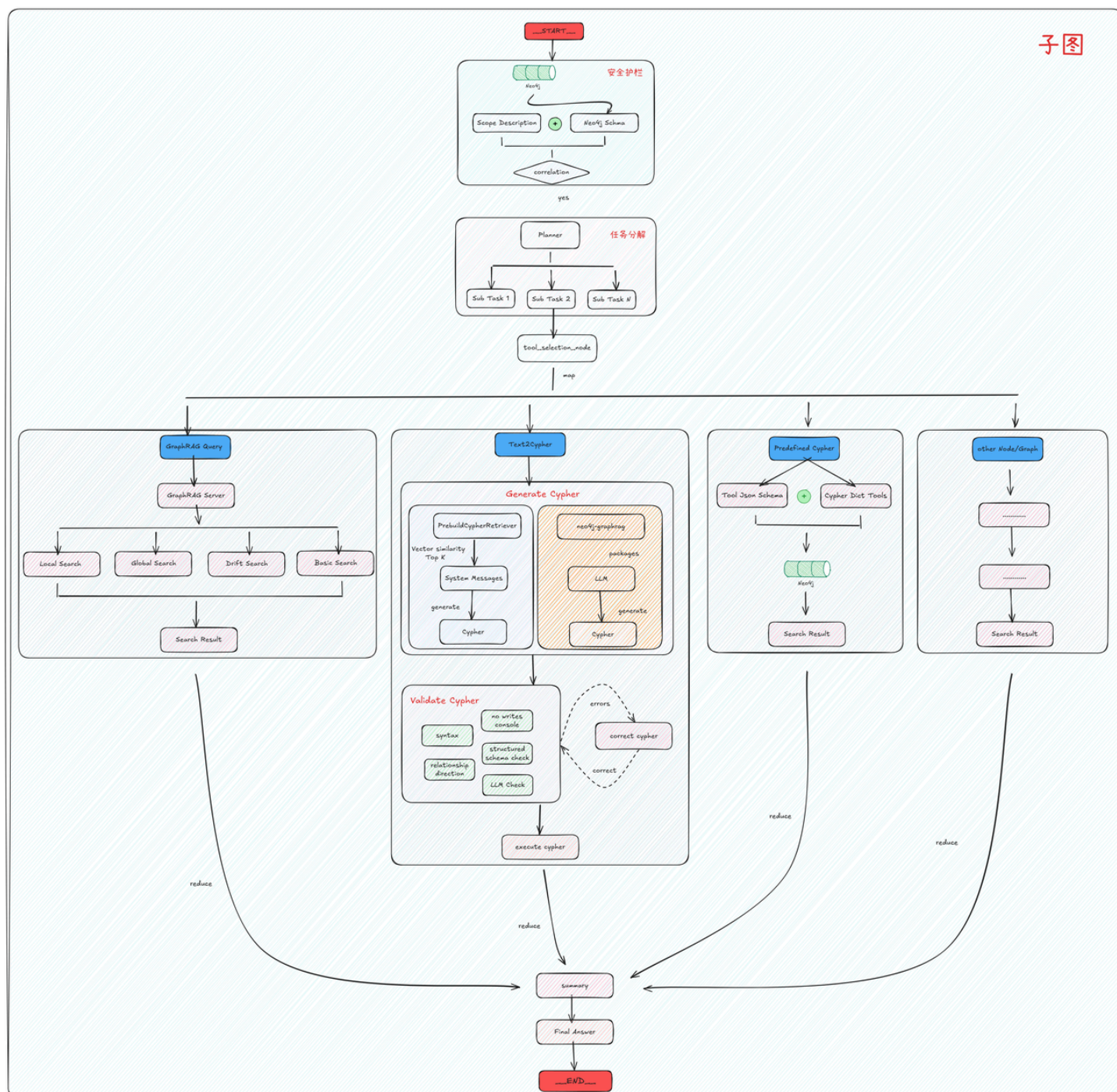
```
1
2  GUARDRAILS_SYSTEM_PROMPT = """
3  你是企业产品信息与订单管理系统中的范围检查组件。
4  你的职责是确定用户的问题是否在系统的处理范围内。
5
6  请遵循以下规则：
7  1. 如果问题与企业商品信息与订单管理相关，包括但不限于：
8     - 商品信息查询
9     - 商品分类信息
10    - 供应商信息
11    - 客户信息
12    - 订单信息和状态
13    - 员工信息
14    - 物流信息
15
16    如果相关，请仅输出："continue"
17
18    2. 如果问题明显与企业商品信息与订单管理无关（如政治、娱乐、天气等），请仅输出："end"
19
20    3. 如产生疑问，请假设问题可能相关，宁可错误地接受也不要错误地拒绝
21
22    4. 在做决定时，请严格按照系统的数据库结构来判断，如果不相关，则不属于经营范围，请输出："end"
23
24    5. 仅输出指定的结果："continue"或"end"
25    """
```

代码块

```
1  # 根据格式化输出的结果，返回不同的响应
2  if guardrails_output.decision == "end":
3      logger.info("-----Fail to pass guardrails check-----")
4      return [{"messages": [AIMessage(content="抱歉，我家暂时没有这方面的商品，可以在别家看看哦~")]}]
5  else:
6      logger.info("-----Pass guardrails check-----")
7      system_prompt = GET_ADDITIONAL_SYSTEM_PROMPT.format(
8          logic=state.router["logic"]
9      )
10     messages = [{"role": "system", "content": system_prompt}] + state.messages
11     response = await model.ainvoke(messages)
```

graphrag-query 处理逻辑

graphrag-query 是智能客服系统中最重要的一个实现逻辑，它会包含 Microsoft GraphRAG 和 Neo4j GraphRAG 的混合检索，根据用户的问题，选择合适本地知识库获取实时的结果。其内部实现非常复杂，因此它并不是一个简单的节点，而是 langGraph 中构建 Multi-Agent 架构的结构：子图（SubGraph）。



同 `get-additional-query` 节点一样，`graphrag-query` 节点也会包含一个安全护栏，用于判断用户的问题是否属于经营范围。如果属于经营范围，则继续执行后续的逻辑，否则直接结束当前会话。因为用户如果最初提问的问题类似：请问我的订单xxxx现在送到哪里了，这种问题包含了详细

的信息，所以不会进入到 `get-additional-query` 去追问补充的信息。这部分的处理逻辑和 `get-additional-query` 节点是一样的，这里就不再赘述了。

接下来，当用户的问题被判断为属于经营范围后，则会实际的去查询对应的工具，比如不同的知识库，但在执行实际的查询之前，我们需要先通过任务分解组件去拆分用户输入进来的问题，因为用户的问题可能包含多个子任务，比如：请问你家的小米音箱和电视都有哪些型号，这明显是两个任务，分别为：小米音箱的型号和小米电视的型号。除此以外，对于多跳的查询，也需要通过任务分解组件去拆分。

在具体的实现上，任务分解组件会根据用户的问题，生成一个任务列表，然后根据任务列表，每一个子任务，根据任务的性质，选择合适的工具去执行，其实现的核心思路是：

代码块

```
1  PLANNER_SYSTEM_PROMPT = """
2  你是一个电商平台智能客服系统中的任务规划组件。
3  你的职责是分析用户的查询，并将其拆分为独立的子任务。
4
5  请遵循以下规则：
6  1. 如果问题可以拆分为多个独立子任务，返回这些子任务列表
7  2. 如果问题很简单，不需要拆分，则返回一个只包含原问题的列表
8  3. 子任务之间应该是独立的，不要相互依赖
9  4. 确保子任务不会返回重复或相似的信息
10 5. 将相互依赖的任务合并为一个问题
11 6. 将返回相同信息任务合并为一个问题
12
13 示例：
14 - 问题: "北风商贸有哪些饮料类产品？它们的价格是多少？"
15 子任务: ["北风商贸有哪些饮料类产品？", "北风商贸饮料类产品的价格是多少？"]
16
17 - 问题: "谁是处理了订单10248的员工？这个订单发往哪里？"
18 子任务: ["谁是处理了订单10248的员工？", "订单10248的配送信息是什么？"]
19
20 - 问题: "供应商Exotic Liquids提供了哪些产品？这些产品的库存情况如何？"
21 子任务: ["供应商Exotic Liquids提供了哪些产品？", "Exotic Liquids供应的产品库存情况如何？"]
22
23 - 问题: "订单10248包含哪些产品？这些产品是哪家供应商提供的？"
24 子任务: ["订单10248包含哪些产品？", "订单10248中的产品分别由哪些供应商提供？"]
25
26 - 问题: "茶叶产品的使用说明手册在哪里可以找到？"
27 子任务: ["茶叶产品的使用说明手册在哪里？"]
28
29 - 问题: "客户对北风商贸巧克力产品的评价如何？"
```

```
30 子任务: ["客户对北风商贸巧克力产品的评价? "]
31  """
```

对拆分出的每一个子任务，替换后续流程中工具调用时接收到的 `query` 参数，即：让接下来的工具调用，执行的查询语句，是根据子任务生成的。伪代码如下所示：

代码块

```
1  planner_task_decomposition = {
2      "next_action": next_action,
3      "tasks": planner_output.tasks
4  or [
5      Task(
6          question=state.get("question", ""),
7          parent_task=state.get("question", ""),
8      )
9  ]
10 }
```

有了子任务后，接下来就是根据子任务的性质，选择合适的工具去执行。因此我们需要用到 `LangGraph` 的条件边，在条件边中，定义都有哪些子任务，以及每个子任务都是做什么的，其实现形式的伪代码如下：

代码块

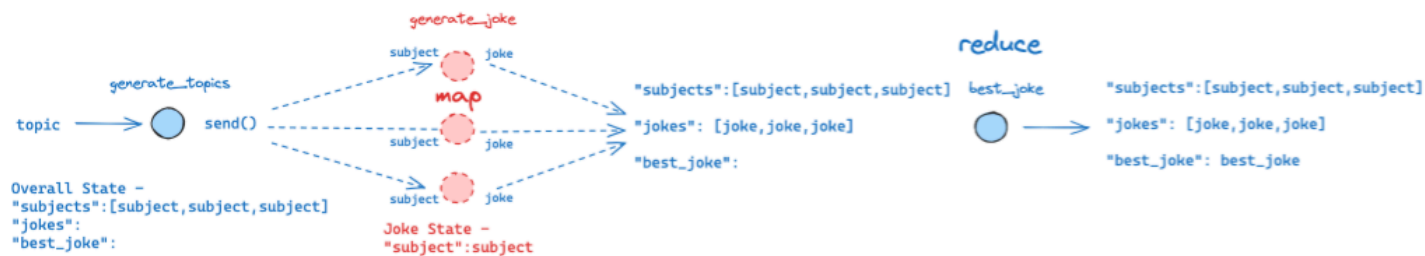
```
1  #%% md
2  ```python
3      async def tool_selection(
4          state: ToolSelectionInputState,
5      ) -> Command[Literal["cypher_query", "predefined_cypher",
6          "customer_tools"]]:
7          """
8          Choose the appropriate tool for the given task.
9          """
10         # 调用工具选择链，生成针对每个任务要调用的工具名称和参数
11         tool_selection_output: BaseModel = await
12         tool_selection_chain.ainvoke(
13             {"question": state.get("question", "")}
14         )
15         # 根据路由到对应的工具节点
16         if tool_selection_output is not None:
17             tool_name: str =
18             tool_selection_output.model_json_schema().get("title", "")
19             tool_args: Dict[str, Any] = tool_selection_output.model_dump()
```

```

18         if tool_name == "predefined_cypher":
19             return Command(
20                 goto=Send(
21                     "predefined_cypher",
22                     {
23                         "task": state.get("question", ""),
24                         "query_name": tool_name,
25                         "query_parameters": tool_args,
26                         "steps": ["tool_selection"],
27                     },
28                 )
29             )
30         elif tool_name == "cypher_query":
31             return Command(
32                 goto=Send(
33                     "cypher_query",
34                     {
35                         "task": state.get("question", ""),
36                         "query_name": tool_name,
37                         "query_parameters": tool_args,
38                         "steps": ["tool_selection"],
39                     },
40                 )
41             )
42
43         else:
44             return Command(
45                 goto=Send(
46                     "customer_tools",
47                     {
48                         "task": state.get("question", ""),
49                         "query_name": tool_name,
50                         "query_parameters": tool_args,
51                         "steps": ["tool_selection"],
52                     },
53                 )
54             )

```

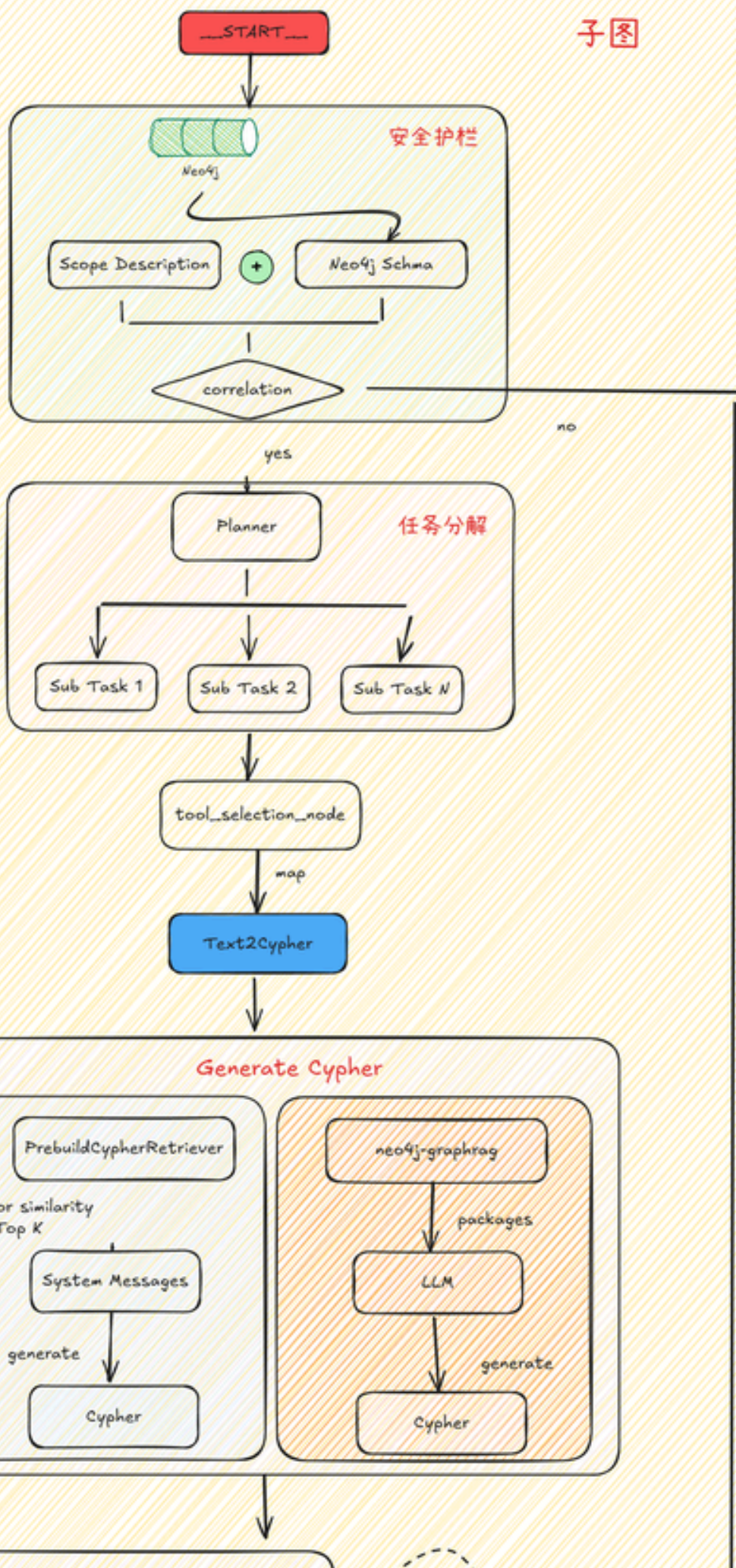
这里面用到的 `Send` 是 `LangGraph` 中的一个 `API`，主要是通过 `Map-Reduce` 对子任务执行并行的处理，然后汇总所有已完成的子任务结果。因为 `Nodes` 和 `Edges` 都是预先定义的，并且基于相同的共享状态进行操作。但对这种动态的执行过程，我们不知道会切分出多少个子任务，也不知道每个子任务会使用哪个工具，所以如果想要多个子任务都能独立应用全局的共享状态，就需要使用 `Map-Reduce` 的执行方式。官方文档：https://langchain-ai.github.io/langgraph/concepts/low_level/#send

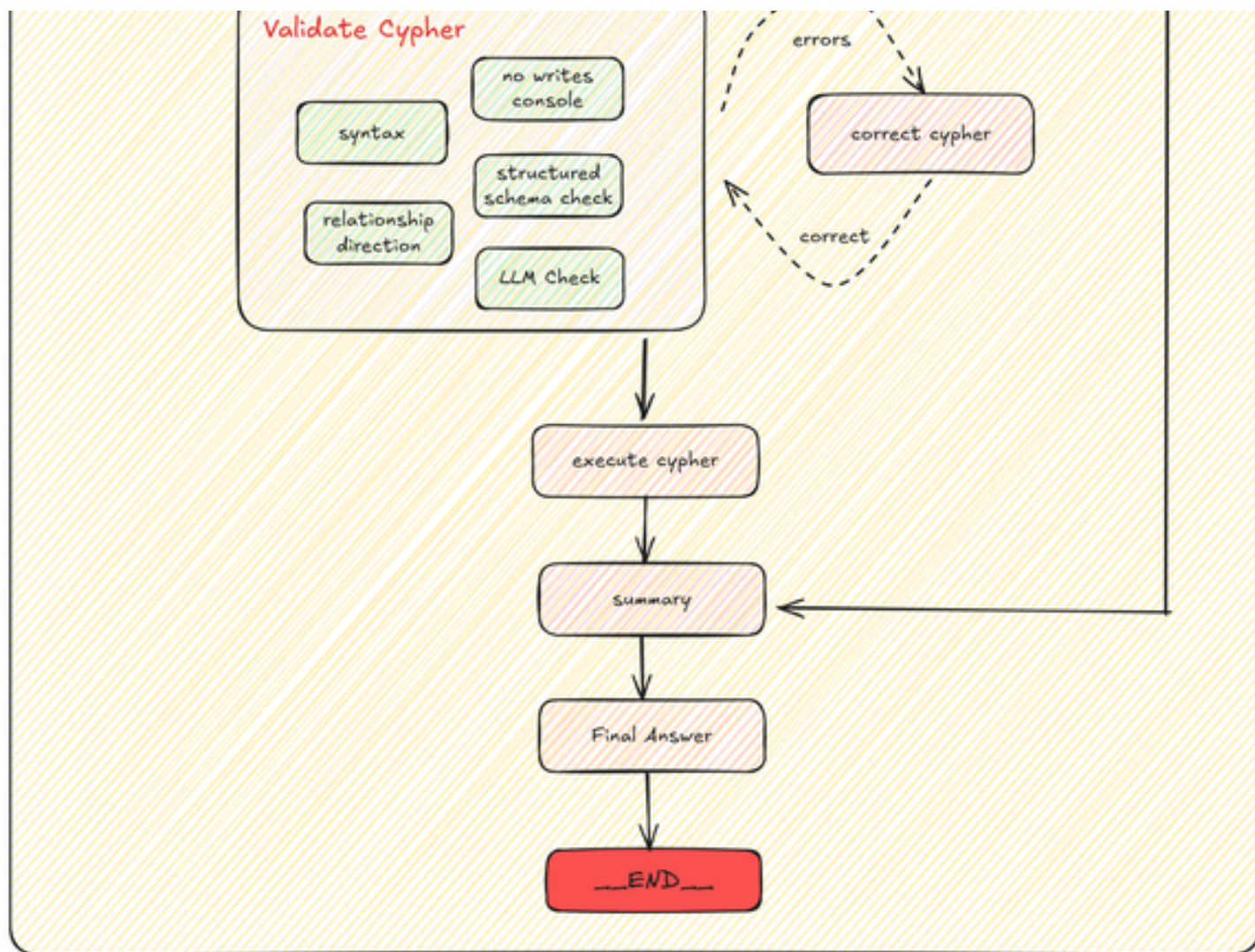


了解了任务分解和分发的流程，接下来我们就要实际定义执行不同子任务的工具节点了。首先来看 `Text2Cypyer` 工具。

Text2Cypyer 工具定义

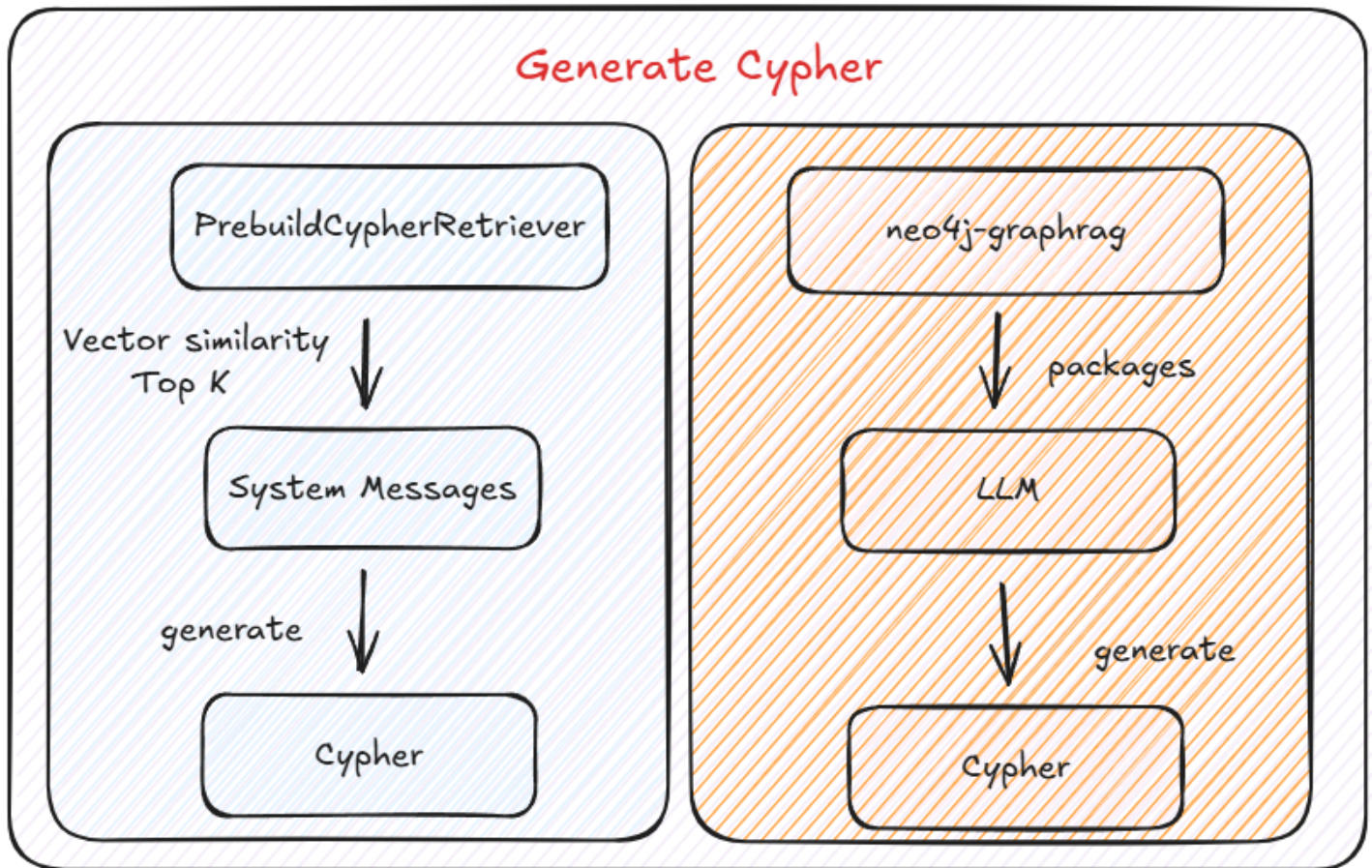
在 `Text2Cypyer` 工具中主要操作的是结构化存储的 `Neo4j` 数据库，因此该工具的核心是：将用户的自然语言问题先转化为 `Cypher` 查询语句，然后再去实际的 `Neo4j` 数据库中执行查询语句，最后将查询结果返回给用户。





关于如何借助大模型根据用户的问题生成较为准确的 Cypher 查询语句，这里核心采用了两种方法，其一是预构建 Cypher 字典，其二是直接借助大模型生成 Cypher 查询语句。如下图所示：

Generate Cypher



预构建 Cypher 字典

预构建 `Cypher` 字典的本质是根据自有数据的情况，人工定义出一些可以正常运行，且能正确返回结果的 `Cypher` 查询语句，这类查询语句可以基于业务类型进行分类，也可以基于数据类型进行分类。（具体以哪种方式分类，取决于自有数据的情况灵活调整）其形式如下：

代码块

```
1 all_examples = {
2     "产品查询": [
3         {
4             "question": "查询所有智能音箱类产品",
5             "cypher": """MATCH (p:Product)-[:BELONGS_TO]->(c:Category)
6 WHERE c.CategoryName = '智能音箱'
7 RETURN p.ProductName, p.UnitPrice, p.UnitsInStock"""
8         },
9         {
10            "question": "查找库存少于20的产品",
11            "cypher": """MATCH (p:Product)
12 WHERE p.UnitsInStock < 20
13 RETURN p.ProductName, p.UnitsInStock
14 ORDER BY p.UnitsInStock"""
15        },
16        {
17            "question": "哪些产品的单价高于5000元?",
18            "cypher": """MATCH (p:Product)
```



```

19 WHERE p.UnitPrice > 5000
20 RETURN p.ProductName, p.UnitPrice
21 ORDER BY p.UnitPrice DESC""""
22     }
23 ],
24     "产品类别": [
25         {
26             "question": "智能家居有哪些产品类别？ ",
27             "cypher": """"MATCH (c:Category)
28 RETURN c.CategoryName, c.Description""""
29         },
30         {
31             "question": "智能灯具类别下有哪些产品？ ",
32             "cypher": """"MATCH (p:Product)-[:BELONGS_TO]->(c:Category)
33 WHERE c.CategoryName = '智能灯具'
34 RETURN p.ProductName, p.UnitPrice""""
35         }
36 ],
37     "供应商相关": [
38         {
39             "question": "供应商小米智能家居提供了哪些产品？ ",
40             "cypher": """"MATCH (p:Product)-[:SUPPLIED_BY]->(s:Supplier)
41 WHERE s.CompanyName = '小米智能家居'
42 RETURN p.ProductName, p.QuantityPerUnit, p.UnitPrice""""
43         },
44         {
45             "question": "中国供应商提供了哪些产品？ ",
46             "cypher": """"MATCH (p:Product)-[:SUPPLIED_BY]->(s:Supplier)
47 WHERE s.Country = '中国'
48 RETURN s.CompanyName, p.ProductName, p.UnitPrice""""
49         }
50 ],
51     "订单查询": [
52         {
53             "question": "订单1001包含哪些产品？ ",
54             "cypher": """"MATCH (o:Order)-[:CONTAINS]->(p:Product)
55 WHERE o.OrderID = 1001
56 RETURN p.ProductName, p.UnitPrice, o.OrderDate""""
57         },
58         {
59             "question": "谁处理了订单1001？ ",
60             "cypher": """"MATCH (o:Order)<-[:PROCESSED]-(e:Employee)
61 WHERE o.OrderID = 1001
62 RETURN e.FirstName, e.LastName, e.Title""""
63         },
64         {
65             "question": "客户AB123下了哪些订单？ ",

```

```

66         "cypher": ""MATCH (o:Order)<-[[:PLACED]]-(c:Customer)
67 WHERE c.CustomerID = 'AB123'
68 RETURN o.OrderID, o.OrderDate, o.ShippedDate
69 ORDER BY o.OrderDate DESC""
70     }
71 ],

```

除了人工构建，另外一种高效的方法是借助大模型自动化生成 **Cypher** 查询语句。如果采用这种方式，需要给到大模型的核心信息就是 **Neo4j** 的 **Schema** 信息，如下格式所示：

代码块

```

1  ```python
2
3      如下是我的 Neo4j 数据库中的 节点和关系情况：
4
5      - 节点类型
6          - Product - 产品节点
7          - Category - 产品类别节点
8          - Supplier - 供应商节点
9          - Customer - 客户节点
10         - Employee - 员工节点
11         - Shipper - 物流公司节点
12         - Order - 订单节点
13
14     - 边类型
15         - BELONGS_TO - 产品属于类别
16         - SUPPLIED_BY - 产品由供应商提供
17         - PLACED_BY - 订单由客户下单
18         - PROCESSED_BY - 订单由员工处理
19         - SHIPPED_VIA - 订单通过物流公司配送
20         - CONTAINS - 订单包含产品
21         - REPORTS_TO - 员工上下级关系
22
23
24     <style>
25     .center
26     {
27     width: auto;
28     display: table;
29     margin-left: auto;
30     margin-right: auto;
31     }
32     </style>
33
34     <p align="center"><font face="黑体" size=4>节点类型</font></p>

```

```

35     <div class="center">
36
37         | 节点类型      | 标识          | 属性
38         |-----|-----|-----
39         | Product      | ProductID     | ProductName, QuantityPerUnit, UnitPrice,
UnitsInStock, UnitsOnOrder, ReorderLevel, Discontinued |
40         | Category      | CategoryID    | CategoryName, Description
41         | Supplier      | SupplierID    | CompanyName, ContactName, ContactTitle,
Address, City, Region, PostalCode, Country, Phone, Fax, HomePage |
42         | Customer      | CustomerID    | CompanyName, ContactName, ContactTitle,
Address, City, Region, PostalCode, Country, Phone, Fax |
43         | Employee      | EmployeeID    | LastName, FirstName, Title, BirthDate,
HireDate, Address, City, Country, HomePhone, Notes |
44         | Shipper       | ShipperID     | CompanyName, Phone
45         | Order         | OrderID       | OrderDate, RequiredDate, ShippedDate,
Freight, ShipName, ShipAddress, ShipCity, ShipRegion, ShipPostalCode,
ShipCountry |
46
47     </div>
48
49     <style>
50     .center
51     {
52     width: auto;
53     display: table;
54     margin-left: auto;
55     margin-right: auto;
56     }
57     </style>
58
59     <p align="center"><font face="黑体" size=4>关系类型</font></p>
60     <div class="center">
61
62         | 边类型          | 源节点        | 目标节点      | 数据源
63         |-----|-----|-----|-----
64         | BELONGS_TO      | Product       | Category      | products.csv (ProductID ->
CategoryID)
65         | SUPPLIED_BY     | Product       | Supplier      | products.csv (ProductID ->
SupplierID)
66         | PLACED_BY       | Customer      | Order         | orders.csv (CustomerID ->
OrderID)

```



```

67 | PROCESSED_BY | Employee | Order | orders.csv (EmployeeID ->
    | OrderID)
68 | SHIPPED_VIA | Order | Shipper | orders.csv (OrderID ->
    | ShipVia)
69 | CONTAINS | Order | Product | order_details.csv (OrderID -
    > ProductID)
70 | REPORTS_TO | Employee | Employee | employees.csv (EmployeeID -
    > ReportsTo)
71 </div>
72
73
74

```

请基于以上信息，按照业务场景进行分类，构建出对应的 Cypher 字典， 并给出对应的 Cypher 查询语句。输出形式如下：

```

76
77 "产品查询": [
78     {
79         "question": "查询所有智能音箱类产品",
80         "cypher": """"MATCH (p:Product)-[:BELONGS_TO]->(c:Category)
81 WHERE c.CategoryName = '智能音箱'
82 RETURN p.ProductName, p.UnitPrice, p.UnitsInStock""""
83     },
84     {
85         "question": "查找库存少于20的产品",
86         "cypher": """"MATCH (p:Product)
87 WHERE p.UnitsInStock < 20
88 RETURN p.ProductName, p.UnitsInStock
89 ORDER BY p.UnitsInStock""""
90     },
91     {
92         "question": "哪些产品的单价高于5000元？ ",
93         "cypher": """"MATCH (p:Product)
94 WHERE p.UnitPrice > 5000
95 RETURN p.ProductName, p.UnitPrice
96 ORDER BY p.UnitPrice DESC""""
97     }
98 ],
99 "产品类别": [
100     {
101         "question": "智能家居有哪些产品类别？ ",
102         "cypher": """"MATCH (c:Category)
103 RETURN c.CategoryName, c.Description""""
104     },
105     {
106         "question": "智能灯具类别下有哪些产品？ ",
107         "cypher": """"MATCH (p:Product)-[:BELONGS_TO]->(c:Category)
108 WHERE c.CategoryName = '智能灯具'

```

```

109         RETURN p.ProductName, p.UnitPrice""
110     }
111 ],
112
113 ```

```

除此以外，还可以定期从业务日志中进行数据抽取，以不断的丰富 **Cypher** 字典，同时存储到向量数据库中，通过向量检索的方式进行 **Cypher** 字典的长期更新和维护以及使用，获取有效 **Cypher** 的方案非常多样。而不论采用哪种方案，虽然此方案需要人工介入，但综合看，一个高质量的预构建 **Cypher** 字典用于大模型生成实时需求 **Cypher** 的 **Few-shot** 效果非常稳定同时准确率较高。结合 **Few-shot** 的完整提示模版如下所示：

代码块

```

1      [
2          (
3              "system",
4              (
5                  "根据输入的问题，将其转换为Cypher查询语句。不要添加任何前言。"
6                  "不要在响应中包含任何反引号或其他标记。注意：只返回Cypher语句！"
7              ),
8          ),
9          (
10             "human",
11             (
12                 """"你是一位Neo4j专家。根据输入的问题，创建一个语法正确的Cypher查询语
13                 句。
14
15                 不要在响应中包含任何反引号或其他标记。只使用MATCH或WITH子句开始查
16                 询。只返回Cypher语句！
17
18                 以下是数据库模式信息：
19                 {schema}
20
21                 下面是一些问题和对应Cypher查询的示例：
22                 {fewshot_examples}
23
24                 用户输入： {question}
25                 Cypher查询：""""
26             ),
27         ],
28     )

```

借助大模型生成 Cypher 语句

如果想完全脱离人工，全部交由大模型生成 Cypher 语句，这里提供一个借助 Neo4j 的 neo4j-graphrag 工具自动化生成 Cypher 语句的示例：对应的代码如下：

代码块

```
1  pip install neo4j-graphrag
2
3  from neo4j_graphrag.retrievers import Text2CypherRetriever
4  from neo4j_graphrag.llm import OpenAILLM
5  import time
6  import pandas as pd
7  from neo4j import GraphDatabase
8
9  NEO4J_URI="bolt://localhost"
10 NEO4J_USERNAME="neo4j"
11 NEO4J_PASSWORD="Snowball2019"
12 NEO4J_DATABASE="neo4j"
13
14 driver = GraphDatabase.driver(
15     NEO4J_URI,
16     auth=(NEO4J_USERNAME, NEO4J_PASSWORD)
17 )
18
19 # 这里可以填写 DeepSeek 模型
20 client = OpenAILLM(api_key="sk-7affff34430",
21     base_url="https://api.deepseek.com", model_name='deepseek-chat')
22
23 # 定义用户输入：
24 examples = [
25     "USER INPUT: 'Which actors starred in the Matrix?' QUERY: MATCH (p:Person)-[:ACTED_IN]->(m:Movie) WHERE m.title = 'The Matrix' RETURN p.name"
26 ]
27
28 # 初始化检索器
29 retriever = Text2CypherRetriever(
30     driver=driver,
31     llm=client,
32     neo4j_schema=neo4j_schema, # 可以通过
33     retrieve_and_parse_schema_from_graph_for_prompts 获取动态的Schema
34     examples=examples,
35 )
36
37 # 执行检索：
38 query_text = "muyu 都有哪些朋友？"
```

上述生成 `Cypher` 语句的方法大家可以自行尝试。

其实无论采用哪种生成 `Cypher` 语句的方案，其最终生成的 `Cypher` 语句都可能存在问题，因此一个比较健壮的流程是需要包含**校验-自我纠正-执行-反馈**的完整闭环，所以我本项目中就提供了一套完整的 `Workflow` 的实现思路，其中对于校验，我们提供了多种校验策略，包括：

1. **语法校验**：通过 `EXPLAIN` 关键字，用于分析和展示 `Cypher` 查询的执行计划，而不实际执行该查询，对语法进行校验；
2. **权限控制**：一般提供给大模型使用的 `Cypher` 语句，会限制一些权限，比如：不能删除数据，不能更改表结构等。所以这一步很关键；
3. **关系方向校正**：在图数据库中，关系是有方向性的。例如，`(a)-[:RELATION]->(b)` 表示从节点 a 到节点 b 的关系。如果查询中使用了错误的方向，会导致查询结果不准确。借助 `langchain_neo4j` 的 `CypherQueryCorrector` 来校验 `Cypher` 语句的语法，比如：
`MATCH (a:Person)-[r:FRIENDS_WITH]->(b:Person)`，如果 `r:FRIENDS_WITH` 是反向的，则会被纠正为：`MATCH (a:Person)-[r:FRIENDS_WITH]->(b:Person)`
4. **更高阶**：使用大模型辅助校验 `Cypher` 语句的语法，主要是用来检查查询中涉及的节点和属性是否存在 `Neo4j` 数据库中。存在。
 - 获取动态的 `Neo4j Schema`，构建 `Pydantic` 模型，输出针对当前 `Cypher` 语句的 `error` 和 `mapping` 的错误信息
 - 如果存在 `error`，直接记录
 - 如果存在 `mapping` 的错误，先对字符串类型进行映射检查，构建一个 `Cypher` 查询，检查数据库中是否存在具有指定属性值的节点。（因为 `Neo4j` 重要的属性（如名称、ID、标签等）通常是字符串类型，而用户的查询基本都是字符串类型），

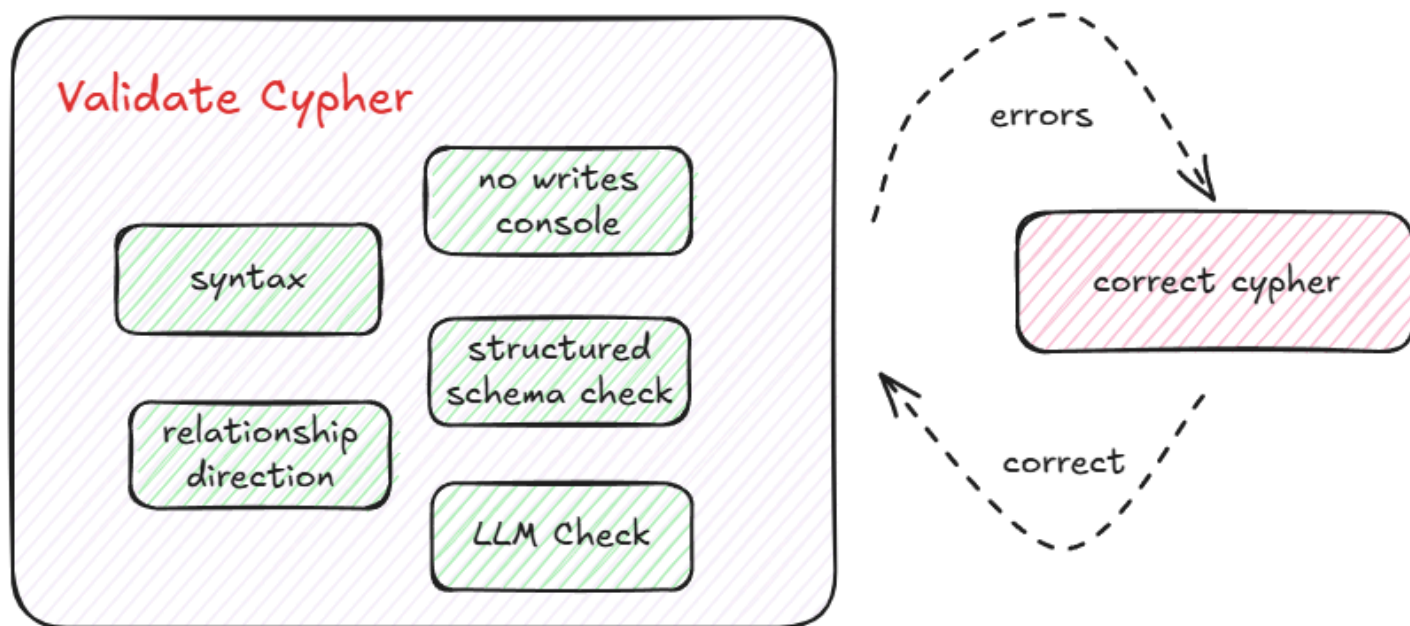
`Mapping` 会表明你的 `Cypher` 查询语法是正确的，但查询中使用的具体值在数据库中不存在。这是数据不存在的问题，而不是查询语法的问题。比如其处理形式如下所示：`Map:`

```
mapping_errors: ['Missing value mapping for Order on property orderId with value 12345', 'Missing value mapping for Product on property ProductName with value 小米音箱']
```

所以这里有很多策略，比较常用的就是：

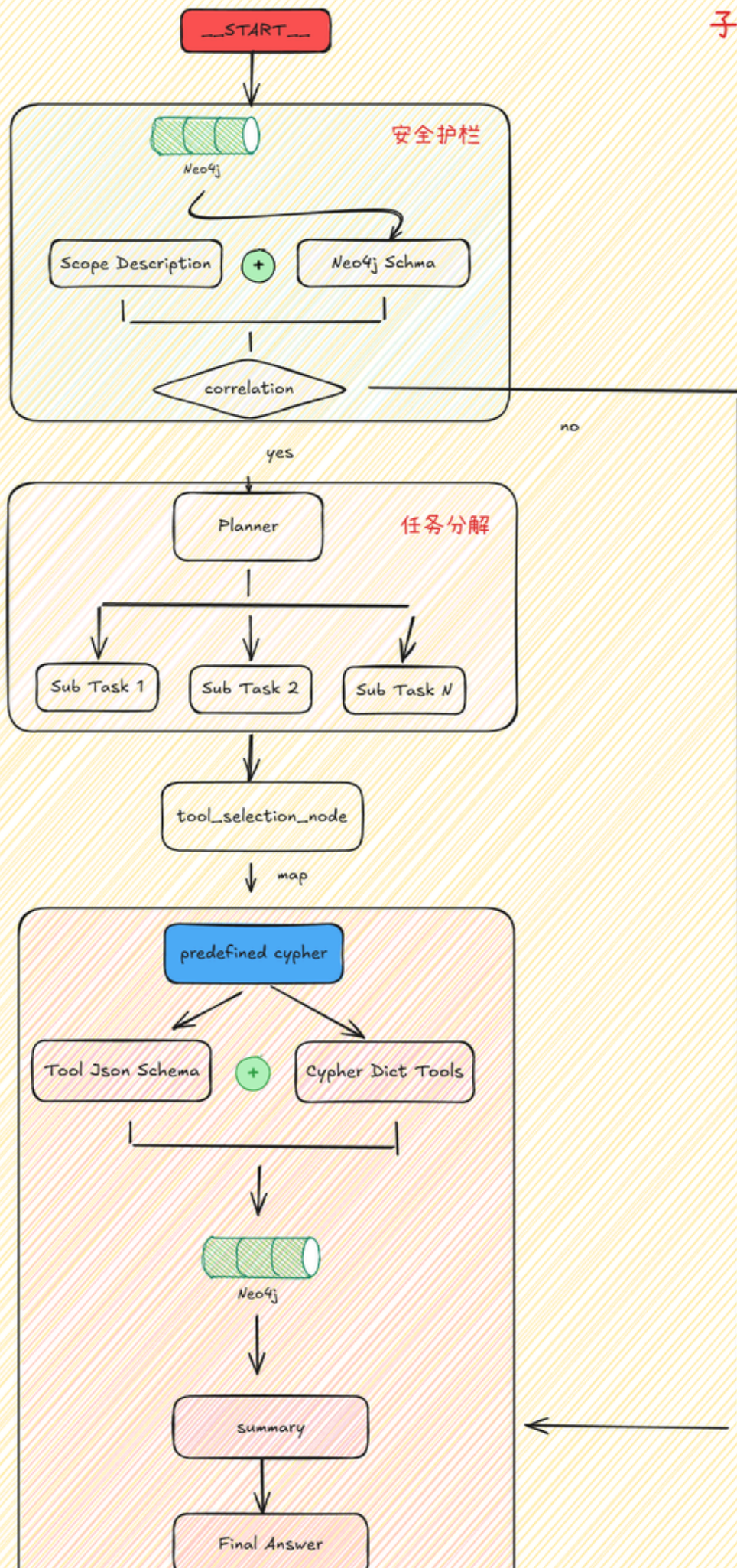
1. 如果存在 `error`，可以直接终止对话，也可以创建一个自修正节点进行 `Cypher` 的自我修复，然后再执行实际的查询
2. 如果存在 `mapping`，则说明用户的问题在数据库中不存在：
 - a. 可以重新进入提问状态，与用户确认信息，或者要求提供更多的信息
 - b. 根据历史会话等再次重新生成 `Cypher` 语句

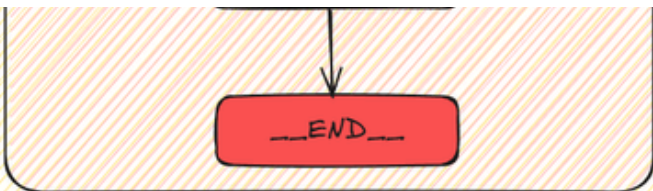
c. 直接结束，直接告诉用户没有查询到相关信息



预构建的 Cypher 工具节点

Text2Cypher 是通过大模型生成 Cypher 语句的，但是大模型生成的 Cypher 语句可能存在一些问题而且耗时较长，所以可以预构建一个 Cypher 工具节点，这个预构建的 Cypher 工具节点与 Text2Cypher 的 Cypher 字典不同，Text2Cypher 的 Cypher 字典是用来作为 Few-shot 的示例填充到提示模版中，从而引导大模型生成正确的 Cypher 语句，而预构建的 Cypher 工具节点是用来直接获取到对应的 Cypher 语句进行执行，它是作为工具节点来使用的。如下图所示：





END

代码块

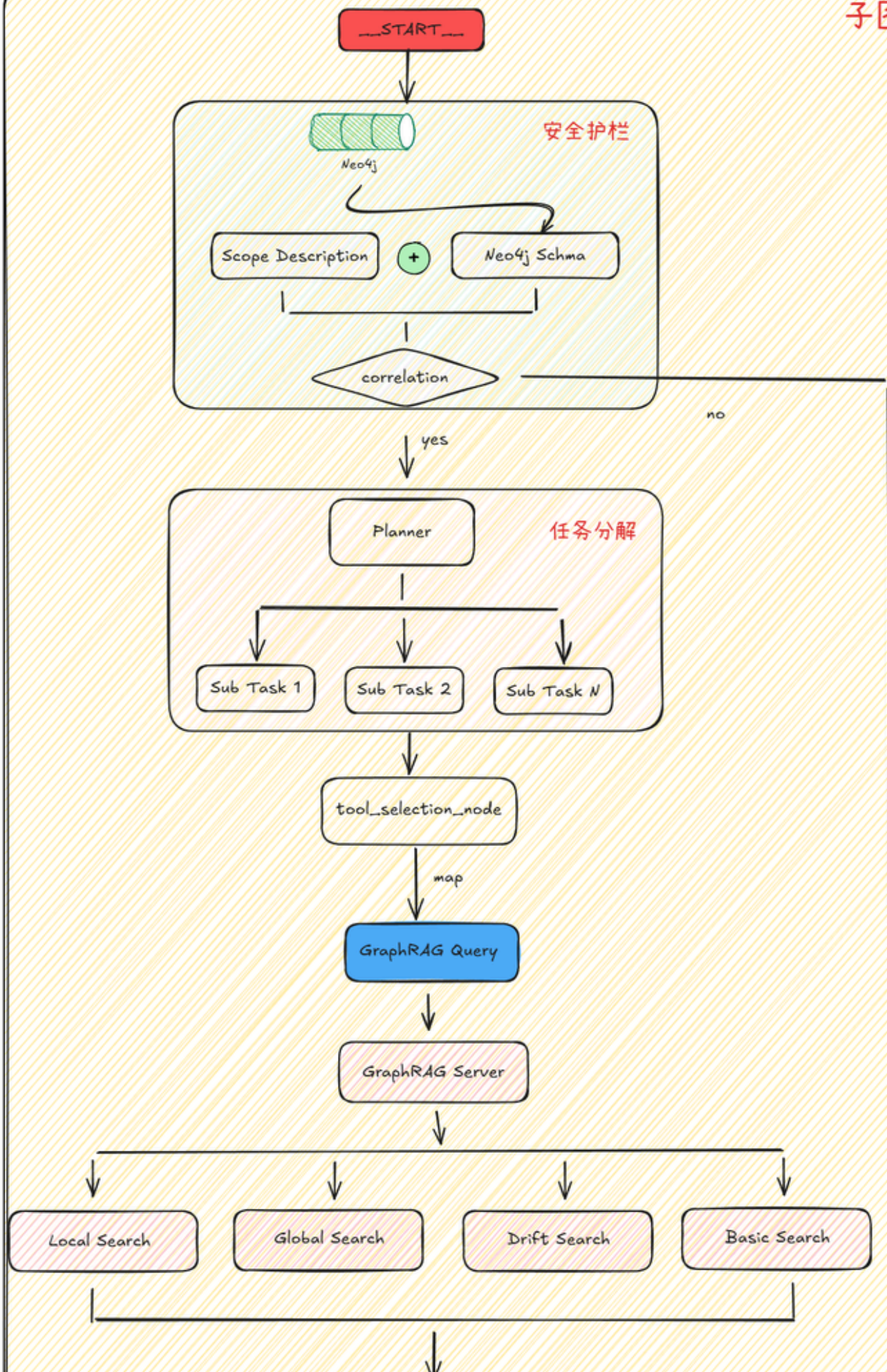
```
1 predefined_cypher_dict: Dict[str, str] = {
2     # 产品类查询
3     "product_by_name": "MATCH (p:Product) WHERE p.ProductName CONTAINS
    $product_name RETURN p.ProductName, p.UnitPrice, p.UnitsInStock,
    p.CategoryName",
4     "product_by_category": "MATCH (p:Product)-[:BELONGS_TO]->(c:Category)
    WHERE c.CategoryName = $category_name RETURN p.ProductName, p.UnitPrice,
    p.UnitsInStock",
5     "product_by_supplier": "MATCH (p:Product)-[:SUPPLIED_BY]->(s:Supplier)
    WHERE s.CompanyName = $supplier_name RETURN p.ProductName, p.UnitPrice,
    p.UnitsInStock",
6     "products_low_stock": "MATCH (p:Product) WHERE toInteger(p.UnitsInStock) <
    10 RETURN p.ProductName, p.UnitsInStock, p.CategoryName ORDER BY
    toInteger(p.UnitsInStock)",
7     "products_popular": "MATCH (p:Product)<-[:ABOUT]-(r:Review) RETURN
    p.ProductName, count(r) as ReviewCount, avg(toFloat(r.Rating)) as AvgRating
    ORDER BY ReviewCount DESC LIMIT 10",
8
9     # 客户类查询
10    "customer_by_name": "MATCH (c:Customer) WHERE c.CompanyName CONTAINS
    $customer_name RETURN c.CompanyName, c.ContactName, c.Phone, c.Country",
11    "customer_orders": "MATCH (c:Customer)-[:PLACED]->(o:Order) WHERE
    c.CompanyName = $customer_name RETURN o.orderId, o.OrderDate, o.ShippedDate",
12    "customer_purchase_history": "MATCH (c:Customer)-[:PLACED]->(o:Order)-
    [:CONTAINS]->(p:Product) WHERE c.CompanyName = $customer_name RETURN
    p.ProductName, o.OrderDate, p.UnitPrice",
13
14    # 订单类查询
15    "order_by_id": "MATCH (o:Order) WHERE o.orderId = $order_id RETURN
    o.OrderDate, o.RequiredDate, o.ShippedDate, o.CustomerName",
16    "order_details": "MATCH (o:Order)-[contains:CONTAINS]->(p:Product) WHERE
    o.orderId = $order_id RETURN p.ProductName, contains.Quantity,
    contains.UnitPrice, toFloat(contains.Quantity) * toFloat(contains.UnitPrice)
    as TotalPrice",
17    "recent_orders": "MATCH (o:Order) RETURN o.orderId, o.OrderDate,
    o.CustomerName ORDER BY o.OrderDate DESC LIMIT 10",
18    "delayed_orders": "MATCH (o:Order) WHERE o.RequiredDate < o.ShippedDate OR
    (o.RequiredDate < date() AND o.ShippedDate IS NULL) RETURN o.orderId,
    o.OrderDate, o.RequiredDate, o.ShippedDate, o.CustomerName",
19
```

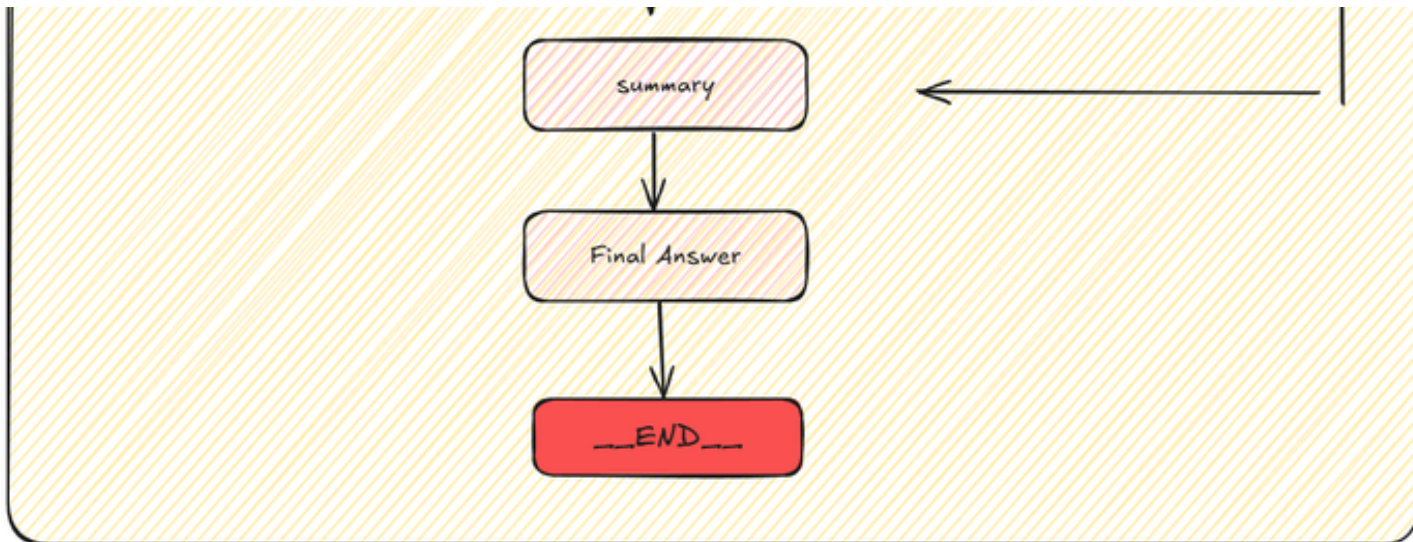
```
20      # 供应商类查询
21      "supplier_by_country": "MATCH (s:Supplier) WHERE s.Country = $country
RETURN s.CompanyName, s.ContactName, s.Phone",
22      "supplier_products": "MATCH (s:Supplier)<-[:SUPPLIED_BY]-(p:Product) WHERE
s.CompanyName = $supplier_name RETURN p.ProductName, p.UnitPrice,
p.UnitsInStock",
23
24      # 类别类查询
25      "all_categories": "MATCH (c:Category) RETURN c.CategoryName,
c.Description",
26      "category_products": "MATCH (c:Category)<-[:BELONGS_TO]-(p:Product) WHERE
c.CategoryName = $category_name RETURN p.ProductName, p.UnitPrice,
p.UnitsInStock",
27      "category_product_count": "MATCH (c:Category)<-[:BELONGS_TO]-(p:Product)
RETURN c.CategoryName, count(p) as ProductCount ORDER BY ProductCount DESC",
```

预构建的 `Cypher` 工具节点需要按照业务类型，即高频用户查询的类型进行分类后，能够直接获取到对应的 `Cypher` 语句进行执行，从而提高查询效率。

自定义 Microsoft GraphRAG 的检索节点

当通过 `Microsoft GraphRAG` 构建好了离线索引后，直接调用其接口，就可以将自然语言传入到 `GraphRAG` 的 `Workflow` 中进行检索，无需 `Text2Cypher` 的转化过程。该工具的使用方法并不复杂，**核心主要在于离线索引的构建阶段。**





图像识别节点

在智能客服系统中，上传图片是用户非常常见的一种交互方式，但这个过程只需要接入视觉大模型，就可以将图片中的文字识别出来，从而转化为自然语言，然后就可以进行后续的问答流程。整个过程并不复杂，因此该工具仅需要一个普通的 `LangGraph` 节点即可快速实现，核心代码如下：

代码块

```
1  """ md
2  ```python
3      payload = {
4          "model": vision_model,
5          "messages": [
6              {
7                  "role": "system",
8                  "content": "你是一个专业的图像分析助手。请详细分析图片中的内容，特别关注产品细节、品牌、型号等信息。"
9              },
10             {
11                 "role": "user",
12                 "content": [
13                     {
14                         "type": "image_url",
15                         "image_url": {
16                             "url": f"data:image/jpeg;base64,{image_data}"
17                         }
18                     }
19                 ]
20             }
21         ],
22         "max_tokens": 4000,
23         "temperature": 0.7
24     }
25
26     # 发送API请求
```



```

27         async with aiohttp.ClientSession() as session:
28             async with session.post(
29                 f"{base_url}/chat/completions",
30                 headers=headers,
31                 json=payload,
32                 timeout=60 # 增加超时时间
33             ) as response:
34                 if response.status == 200:
35                     result = await response.json()
36                     image_description = result["choices"][0]["message"]
37                     ["content"]
38                     logger.info(f"Successfully processed image and generated
39                     description")
40
41                     # 使用图片描述和用户问题生成最终回复
42                     # 从lg_prompts导入电商客服模板
43
44                     # 构建回复请求
45                     if settings.AGENT_SERVICE == ServiceType.DEEPSEEK:
46                         model =
47                         ChatDeepSeek(api_key=settings.DEEPSEEK_API_KEY,
48                         model_name=settings.DEEPSEEK_MODEL, temperature=0.7, tags=["image_query"])
49                     else:
50                         model = ChatOllama(model=settings.OLLAMA_AGENT_MODEL,
51                         base_url=settings.OLLAMA_BASE_URL, temperature=0.7, tags=["image_query"])
52                     # 使用专门的图片查询提示模板
53                     system_prompt = GET_IMAGE_SYSTEM_PROMPT.format(
54                         image_description=image_description
55                     )
56                     messages = [{"role": "system", "content": system_prompt}]
57                     + state.messages
58
59                     response = await model.ainvoke(messages)
60                     return {"messages": [response]}
61
62                 else:
63                     error_text = await response.text()
64                     logger.error(f"Vision API Request Failed:
65                     {response.status} - {error_text}")
66                     return {"messages": [AIMessage(content=f"抱歉，我无法查看这张
67                     图片，请重新上传。")]}
```

该工具节点主要用于将图片中的文字识别出来，然后转化为自然语言，从而进行后续的问答流程。

总结

以上就是灵犀中，`LangGraph` 的工具节点定义的全部内容，可以基于自己的业务数据情况进行高效的二次开发。